

CA Exercise 1 – Campsite Management System

The objective of this CA exercise is to create a campsite management system in Java that makes heavy use of custom-built internal data structures. The system should allow the user to manage multiple campsites. Every campsite consists of multiple camping areas. Camping areas consist of multiple camping plots/pitches. A plot/pitch can have a static dwelling on it (static mobile home, bungalow, apartment, or similar) or just be an empty plot/pitch which can be used for a tent, caravan, campervan, motorhome, or similar. Bookings for stays/vacations are made for specific camping areas, and bookings are automatically assigned a suitable available plot/pitch within the camping area. Bookings are made by specific guests. Guests can have multiple bookings across multiple campsites.

In short, there can be many campsites, a campsite can have many camping areas, a camping area can have many plots/pitches, a booking is made for a guest for a specific plot/pitch. A guest can have many bookings (complete and incomplete).

The system should allow the user to:

- Add a new campsite to the system. The following information should be stored for each campsite: name, address, number of stars, and contact information. It should also be possible to record facilities available at the campsite, such as: restaurants, swimming pools, shops, spa, gym, WiFi, laundry, medical centre, etc.
- Add a new camping area to a campsite. The following information should be stored for each camping area: camping area identifier (a string e.g. "Lake View Area 1"), and description.
 - Note that the camping area identifier must be unique for the campsite, so check that no other camping area in the same campsite has the same identifier when creating a new camping area.
- Add a new plot/pitch to a camping area. The following information should be stored for each plot/pitch: identifier (string), type (static mobile home, bungalow, apartment, empty pitch, etc.), whether the plot/pitch has power (yes for static; optional for others), how many bedrooms and beds (if a static dwelling), size (in square metres), facilities (e.g. air conditioning), cost per night (€), and a photo (URL) of the plot/pitch.
- Add a guest to the system. Properties that should be stored are: guest identifier (e.g. integer/serial number), name, date of birth, some contact information (e.g. address, telephone, email, etc.), and any pertinent ancillary information.
- Add a booking to a given camping area. Properties that should be stored are: booking dates (start and end), camping area identifier, guest identifier, and information regarding boomer preferences e.g. type (bungalow, or empty pitch), number of guests, powered or not, min size, minimum bedrooms (if applicable), and budget (€). The system should search all plots/pitches in the specified camping area and identify one that is available and matches all criteria. The booking is then made/recorded for that specific plot/pitch. If no suitable plot/pitch can be identified then the user is informed of this.

- Cancel a booking. This will remove the pending booking record from the appropriate plot/pitch of the appropriate campsite camping area.
- View all pending bookings in both (1) a given campsite (organised first by camping area and then by plot/pitch), and (2) the overall system (organised by campsite, camping area, and plot/pitch). Appropriate booking details (plot/pitch, guests, dates, etc.) and totals should be displayed.
- View a given guest's record by identifier. This will also show details of all pending and past bookings.
- Interactively "drill down" through campsites, camping areas, plots/pitches, and bookings using an appropriate GUI.
- A "smart find" facility for camping holidays. Given appropriate guest information (including number of guests, budget, min stars, desired dates, required/desired facilities (e.g. pool and air conditioning), etc.) the system will identify a suitable campsite, camping area, plot/pitch and date to make a booking.
 - You can have the "smart find" behave any way you think appropriate.
- Remove a plot/pitch. Any pending bookings for the removed plot/pitch should be moved/reassigned (if possible) to another suitable plot/pitch in any camping area (or camping areas) in the same campsite.
- Reset facility. Clears all system data.
- Save and load the entire system data to support persistence between executions.
 - This can be done using any suitable file format (e.g. CSV, XML, binary, etc.). There is no need to use any database system beyond this.
- Other appropriate facilities for the campsite management system as you see fit. Be mindful of the marking scheme below, though, as this is what you will be marked against.

Notes

- This is an individual CA exercise, and students should work by themselves to complete it.
- You will have to demonstrate this CA exercise in the lab sessions (or via Zoom) and you will be interviewed on various aspects of it.
 - You are expected to be able to answer all questions on any code you are forwarding as your own. An interview weighting will be applied to reduce your overall mark if you cannot adequately answer all questions in the demo/interview (so prepare for the demo by reviewing the code and be ready to answer questions on any aspect of it).
- This CA exercise is worth 35% of your overall module mark.
- You should implement a suitable JavaFX graphical user interface to interact with your system.
 - The GUI does not have to be particularly fancy but should nevertheless be functional.
- You should implement a set of JUnit classes to systematically test your code as you develop it. Exactly what you test for is up to you but make sure that you demonstrate a test-driven approach to your development.
- You should also use appropriate tooling (such as Git and a repository such as Github) to manage your code and its development.

- It is important to note that you cannot use any existing Java collections or data structures classes (e.g. ArrayList, LinkedList, or any other class that implements the Collection interface or any of its children – if in doubt, ask me!). You also cannot use regular arrays directly. You essentially have to implement the required data structures and algorithms from scratch and from first principles (in line with the module learning outcomes). This is deliberate as I want you to get your hands dirty.
 - You will have to create numerous custom ADTs such as Campsite, CampingArea, Plot, Booking, Guest, etc. (your classes/class names may differ) and use some form of custom linked-list implementation.
 - You should also avoid simply storing data using the JavaFX components themselves. The JavaFX components can of course be used to display data/information as required, but the main data should be stored separate from the JavaFX components.
- The indicative marking scheme below indicates where you should focus your efforts.

Indicative Marking Scheme

- Appropriate custom ADTs = 10%
- Create/add facilities (campsite, camping area, plot/pitch, guest, booking) = 15%
- Cancel booking = 5%
- View pending bookings – structured view (by campsite and overall) = 8%
- View guest record = 6%
- Interactive drill-down = 8%
- Smart find facility for camping holidays = 8%
- Remove plot/pitch = 6%
- Reset facility = 5%
- Save/load facility = 8%
- Appropriate JavaFX user interface = 6%
- JUnit testing (minimum of 6 useful unit tests) = 5%
- General (commenting, style, logical approach, completeness, tooling, etc.) = 10%